

SUPPLY MANAGEMENT SYSTEM

Task Register

Advanced Warehouse, Logistics & Field Operations

Maps to FSD Addendum 27 | 13 tasks | 38 dev days | 84 test cases

Version 1.0 | 2026

FSD source	FSD Addendum 27 - Advanced Warehouse, Logistics & Field Ops
Total tasks	13
Total dev days	38 days (~8 working weeks)
Critical tasks	2
Total test cases	84
Epics	8 epics across enhancements, new modules, and mobile app
Test data	Dell i7 variant, Cement, and asset 'Bosch Drill' traced across all tasks

1. FSD Traceability Summary

Task	Title	Days	FSD	Pri
WL-001	Barcode generation service, label templates, and batch ...	3d	Sec 2	High
WL-002	Expiry alerts dashboard, near-expiry/expired reports, a...	2d	Sec 3	High
WL-003	CycleCounts and CycleCountLines schema, count sheet gen...	3d	Sec 4	Critical
WL-004	Asset register with assignment history, depreciation, a...	3d	Sec 5	High
WL-005	Multi-vendor per variant with PO integration and reorde...	2d	Sec 6	High
WL-006	Demand forecast engine with SMA/WMA/Exponential Smoot...	3d	Sec 7	Medium
WL-007	FulfillmentOrders, directed PickLines, and pick confirm...	4d	Sec 8.1, 8.2	Critical
WL-008	Wave picking, packing station workflow, and carton trac...	3d	Sec 8.3, 8.4	High
WL-009	Carrier framework, rate shopping, label generation, and...	3d	Sec 9	High
WL-010	3PL inventory ownership and drop-ship fulfillment	2d	Sec 10, 11	Medium
WL-011	TransferOrders with approval workflow and ship/receive ...	3d	Sec 12	High
WL-012	Mobile app: GRN receiving, put-away, picking, and stock...	4d	Sec 13	High
WL-013	Mobile app: cycle counting, transfer receiving, and off...	3d	Sec 13.1 (#17)	Medium
TOTAL	13 tasks	38d		

2. Detailed Task Register

Epic 1 - Barcode Generation & Expiry Enhancement (#2, #3)

Barcode label generation/printing and expiry management enhancements to existing capabilities.

WL-001

High

3d

Sec 2

Barcode generation service, label templates, and batch printing

Build IBarcodeGenerationService supporting Code128/EAN-13/QR output as PNG/ZPL/PDF. Create customizable label templates for products, bins, GRNs. Batch print via web and mobile.

Development tasks	Test cases (with test data)
• IBarcodeGenerationService using ZXing.Net: GenerateBarcode(value, type, format) returning byte[] image or ZPL string • GET /api/barcodes/generate?value={}&format;={png zpl pdf}&type;={code128 ean13 qr} • LabelTemplates entity (7 fields per FSD Sec 2.2): template types PRODUCT/LOCATION/GRN/SHIPPING with ZPL template and dimensions • POST /api/labels/print-batch: accepts array of variant_ids or bin_ids, generates labels in bulk, returns PDF or ZPL stream • React: Print Labels button on Product detail, Variant list, Bin management, GRN detail screens • Seed 4 default label templates per org: Product Label, Bin Label, GRN Receiving Label, Shipping Label	• Generate Code128 barcode for SKU 'DELL-5450-I7-16-512' returns valid PNG image • Generate EAN-13 for barcode '8901234567892' returns valid 13-digit barcode image • Generate QR code for GRN number returns scannable QR • ZPL output for Zebra printer renders correctly (validated via ZPL viewer) • Batch print 50 product labels returns a multi-page PDF with one label per page • Label template customization: org A's product label shows price, org B's does not (different templates)
Acceptance	• All 3 barcode types generate correctly • ZPL output compatible with standard Zebra printers • Batch printing scales to 100+ labels without timeout

WL-002

High

2d

Sec 3

Expiry alerts dashboard, near-expiry/expired reports, and auto-quarantine

Add shelf_life_days to Products, build Hangfire daily expiry scan, near-expiry and expired stock reports, and configurable auto-quarantine for expired batches.

Development tasks	Test cases (with test data)
• Add shelf_life_days (Integer nullable) to Products entity. EF migration: AddShelfLifeDays • Hangfire daily recurring job: ExpiryAlertJob scans InventoryItems WHERE batch expiry_date BETWEEN today AND today + warning_window_days. Warning window configurable in appsettings (default 30 days). • GET /api/reports/expired-stock: batches where expiry_date < today with qty_on_hand > 0. Shows variant, warehouse, batch, expiry, qty, unit_cost, total_value_at_risk • GET /api/reports/near-expiry?days={n}: batches expiring within N days sorted by expiry_date ascending • Auto-quarantine: add quarantine_status (NORMAL/QUARANTINED) to InventoryItems. When expiry triggers, set QUARANTINED if category.auto_quarantine_on_expiry = true. Quarantined items excluded from pick/issue availability. • React: Expiry Dashboard widget on the Inventory home screen showing count of expired and near-expiry items with drill-down links	• Batch with expiry_date = today-1: appears in expired-stock report with correct value_at_risk • Batch with expiry_date = today+15: appears in near-expiry?days=30 but NOT in near-expiry?days=10 • Auto-quarantine: expired batch in a category with auto_quarantine=true has quarantine_status=QUARANTINED and is excluded from GetAvailability • Hangfire job correctly fires daily and generates notifications for near-expiry items • FEFO picking (Addendum 20): quarantined batches are skipped, next non-quarantined batch selected

Acceptance	• Daily scan catches all expired/near-expiry batches • Quarantined items genuinely excluded from availability • Reports provide actionable data with value at risk
------------	--

Epic 2 - Cycle Counting (#4)

Scheduled partial physical counts with variance reconciliation and automatic stock adjustment posting.

WL-003**Critical****3d****Sec 4****CycleCounts and CycleCountLines schema, count sheet generation, and varian**

Create cycle count entities, the count planning API, count sheet generation with frozen system quantities, mobile-friendly count entry, variance calculation, and automatic stock adjustment posting on approval.

Development tasks	Test cases (with test data)
• CycleCounts entity (12 fields per FSD Sec 4.1) and CycleCountLines entity (11 fields per FSD Sec 4.2) in warehouse_schema • EF migration: AddCycleCounting • POST /api/cycle-counts: create count plan with scope_filter JSON defining what to count (locations, categories, ABC class) • GET /api/cycle-counts/{id}/sheet: generates count sheet lines by querying InventoryItems matching the scope. Snapshots system_qty at generation time (frozen, not recalculated on each view). • POST /api/cycle-counts/{id}/lines/{lineId}/count: submit counted_qty per line. System computes variance_qty and variance_pct automatically. • Variance threshold: configurable per org (default: 2% or 5 units, whichever is greater). Lines exceeding threshold flagged as RECOUNT_REQUIRED. • POST /api/cycle-counts/{id}/approve: Inventory Manager approves variances. On approval, for each line with variance: call IInventoryLedgerService.CreateEntryAsync with type=STOCK_ADJUSTMENT, qty = variance_qty. InventoryItem.qty_on_hand adjusted to match counted_qty. MasterProductLedger entry written with source/dest = 'Adjustment: Cycle Count CC-YYYY-NNNNN'. • Hangfire scheduled job: auto-create cycle counts based on configurable schedule (e.g. count Zone A every Monday, Category Electronics every month)	• TEST DATA: Bin BIN-A-01 has variant DELL-5450-I7-16-512 with system qty_on_hand=5. Physical count finds 4. variance_qty=-1, variance_pct=-20%. • Count sheet correctly snapshots system_qty=5 at generation time even if qty changes after generation • Counted_qty=4 with threshold 2%: variance 20% exceeds threshold, line flagged RECOUNT_REQUIRED • After recount confirms 4: approval posts STOCK_ADJUSTMENT ledger entry with quantity_out=1, InventoryItem.qty_on_hand updated to 4 • Positive variance: system=10, counted=12. On approval, STOCK_ADJUSTMENT with quantity_in=2, qty_on_hand updated to 12 • Cycle count for scope {categories:['Laptop']}: only laptop variants appear on the count sheet • Count approval by non-Inventory-Manager returns 403 • Scheduled auto-count for Zone A creates a new PLANNED cycle count every Monday via Hangfire
Acceptance	• System quantities frozen at sheet generation time • Variance threshold correctly flags lines for recount • Stock adjustments post atomically to InventoryItems, Stock Ledger, and Master Product Ledger on approval

Epic 3 - Asset Tracking & Multi-Vendor (#5, #6)

Non-inventory asset register and multi-vendor per product variant.

WL-004**High****3d****Sec 5****Asset register with assignment history, depreciation, and maintenance scheduling**

Create the Assets and AssetAssignmentHistory entities in assets_schema. Build full CRUD, assignment/return workflow, depreciation tracking, and maintenance date alerting.

Development tasks	Test cases (with test data)
- Assets entity (19 fields per FSD Sec 5.1) and AssetAssignmentHistory entity (9 fields per Sec 5.2) in assets_schema - EF migration: AddAssetTracking - CRUD: GET/POST/PUT /api/assets with filters: category, status, assigned_to, warehouse, warranty_expiry_before - POST /api/assets/{id}/assign: creates assignment record, updates asset status to ASSIGNED, records condition_on_assign - POST /api/assets/{id}/return: closes assignment record (sets returned_at, condition_on_return), updates asset status to AVAILABLE - Depreciation: for STRAIGHT_LINE method, $\text{current_value} = \text{purchase_cost} - (\text{purchase_cost} / \text{useful_life_months} * \text{months_since_purchase})$. Recalculated on demand or via monthly Hangfire job. - Maintenance alerts: Hangfire daily job flags assets where $\text{next_maintenance_date} \leq \text{today} + 7 \text{ days}$. Notification sent to assigned user and their manager. - React: Assets screen under Inventory > Assets with list, detail, assignment history timeline, and maintenance calendar	- TEST DATA: Asset 'Bosch Drill SDS-Plus', code='ASSET-DRILL-001', category=TOOLS, purchase_cost=45000, purchase_date=2026-01-01, useful_life=60 months, depreciation=STRAIGHT_LINE - After 6 months: $\text{current_value} = 45000 - (45000/60*6) = 40500$ - Assign to User 'Ahmed' for Project 'Tower Block A': AssetAssignmentHistory created, asset status = ASSIGNED - Return with condition=NEEDS_REPAIR: assignment closed, asset status=IN_MAINTENANCE - Maintenance alert: asset with next_maintenance_date = today+5 triggers notification - Retired asset excluded from 'available for assignment' queries - Asset assignment history shows complete timeline: assigned Jan 15 -> returned Mar 20 -> assigned Apr 1 -> current
Acceptance	- Depreciation calculated accurately for straight-line method - Assignment/return lifecycle tracked with condition recording - Maintenance alerts fire reliably

WL-005**High****2d****Sec 6****Multi-vendor per variant with PO integration and reorder suggestions**

Create VariantSuppliers mapping table. Integrate into PO line entry with supplier suggestion panel and into reorder alert notifications.

Development tasks	Test cases (with test data)
-------------------	-----------------------------

• VariantSuppliers entity (13 fields per FSD Sec 6.1) in suppliers_schema. EF migration: AddVariantSuppliers • CRUD: GET/POST/PUT/DELETE /api/variant-suppliers with filter by variant_id and supplier_id • GET /api/variants/{id}/suppliers: returns all approved suppliers for a variant sorted by is_preferred then vendor_unit_cost ascending • React PO line form enhancement: after selecting a variant, show a supplier suggestion panel listing VariantSuppliers with vendor_part_number, cost, lead_time. Clicking a suggestion auto-fills the PO supplier and unit_price. • Reorder alert enhancement: when stock falls below reorder_point, include preferred supplier name, cost, and lead_time in the alert notification body • Auto-update last_po_date and last_po_price on VariantSuppliers when a PO is created for that variant+supplier combination	• TEST DATA: Variant DELL-5450-I7-16-512 has 2 suppliers: Lenovo Distributors (preferred, cost=135000, lead=7 days, vendor_part='LEN-5450-I7') and Tech Wholesale (cost=140000, lead=14 days, vendor_part='TW-DELL-I7') • GET /api/variants/{id}/suppliers returns Lenovo first (preferred) then Tech Wholesale (sorted by preference then cost) • PO line: selecting Dell i7 variant shows suggestion panel with both suppliers. Clicking Lenovo sets PO supplier and unit_price=135000 • Creating PO for Lenovo+Dell i7 updates VariantSuppliers.last_po_date and last_po_price • Reorder alert for Dell i7 (below reorder point) includes 'Preferred supplier: Lenovo Distributors, Cost: 135,000, Lead time: 7 days' • Marking Tech Wholesale as inactive hides it from the suggestion panel
Acceptance	• Multi-vendor ranking by preferred+cost works correctly • PO integration auto-fills supplier and price from selection • Reorder alerts include actionable supplier info

Epic 4 - Demand Forecasting (#7)

Consumption-based forecast engine with reorder recommendations.

WL-006**Medium****3d****Sec 7****Demand forecast engine with SMA/WMA/Exponential Smoothing and reorder re**

Build IForecastService with three configurable models, Hangfire scheduled recalculation, accuracy tracking, and reorder recommendation generation.

Development tasks	Test cases (with test data)
- ForecastEntries entity (12 fields per FSD Sec 7.2) in inventory_schema. EF migration: AddDemandForecasting - IForecastService.CalculateForecast(variantId, model, periods): queries InventoryLedgerEntries type IN (MATERIAL_ISSUE, SALES_ISSUE) grouped by month, applies the selected model - SMA: average of last N periods. WMA: weighted average with configurable weights (recent periods weighted higher). Exponential Smoothing: alpha-weighted latest actual + (1-alpha) * previous forecast. - Hangfire recurring job: ForecastRecalculationJob runs weekly (configurable). Calculates for all active variants with at least 3 months of consumption history. - POST /api/forecasts/recalculate: Admin-triggered full recalculation - GET /api/forecasts?variantId={id}: returns forecast entries for next 3 periods with confidence interval - Reorder recommendation: for each variant, if forecasted_qty_next_period * safety_factor (default 1.2) > qty_available, set reorder_recommended=true. Include recommended_order_qty = forecasted_qty * safety_factor - qty_available + safety_stock. - Accuracy tracking: after each period ends, fill actual_qty from real consumption data and calculate forecast_accuracy_pct	- TEST DATA: Dell i7 variant consumption last 6 months: Jan=8, Feb=12, Mar=10, Apr=15, May=11, Jun=13. SMA(3) forecast for Jul = (15+11+13)/3 = 13 - WMA with weights [0.5, 0.3, 0.2] for Jul = 13*0.5 + 11*0.3 + 15*0.2 = 12.8 - Exponential Smoothing (alpha=0.3): forecast tracks correctly from initial seed - Variant with only 2 months history: skipped by forecast job (minimum 3 months required) - Reorder recommendation: forecast=13, safety_factor=1.2, qty_available=5, safety_stock=3. reorder_recommended=true, recommended_order_qty = 13*1.2 - 5 + 3 = 13.6 rounded to 14 - After July ends with actual=14: forecast_accuracy_pct = (1 - 14-13 /14) * 100 = 92.9% - Admin recalculation updates all forecasts without duplicating entries for the same period
Acceptance	- All 3 forecast models produce mathematically correct results - Reorder recommendations actionable with supplier info from WL-005 - Accuracy tracking provides feedback for model tuning

Epic 5 - Pick/Pack/Ship & Wave Picking (#8-11)

Full outbound fulfillment workflow: fulfillment orders, directed pick lists, wave picking, packing with carton tracking.

WL-007**Critical****4d****Sec 8.1, 8.2****FulfillmentOrders, directed PickLines, and pick confirmation workflow**

Create the fulfillment and pick entities in logistics_schema. Build fulfillment order creation from MIR/sales, directed pick list generation with bin-level locations resolved by FEFO, and pick confirmation with short-pick handling.

Development tasks	Test cases (with test data)
• FulfillmentOrders entity (13 fields per FSD Sec 8.1) and PickLines entity (11 fields per Sec 8.2) in logistics_schema • EF migration: AddFulfillmentPicking • POST /api/fulfillments: create from source (MIR, Sales Order, Transfer Order). Auto-generates PickLines with pick_location resolved from InventoryItems + Bin assignments using FEFO/FIFO logic. • Pick location resolution: for each fulfillment line, find InventoryItems for that variant at the fulfilling warehouse, sorted by batch expiry_date ascending (FEFO), then by bin location for efficient walking path. Split across multiple bins if one bin has insufficient qty. • POST /api/fulfillments/{id}/pick: confirm pick lines. For each confirmed line: decrement InventoryItem.qty_on_hand at the pick location, write MATERIAL_ISSUE ledger entry. Short-pick (picked_qty < pick_qty) sets pick_status=SHORT_PICKED and creates a backorder line. • GET /api/fulfillments: list with filters: status, warehouse, priority, assigned_picker, date range • React: Pick Queue screen showing pending fulfillments sorted by priority. Picker opens a fulfillment, sees directed pick list with bin locations, confirms each line.	• TEST DATA: Fulfillment FUL-001 for 8 units of Dell i7 variant. Warehouse has: Bin A-01 (3 units, expiry Jun 2027), Bin B-03 (10 units, expiry Sep 2027). Pick list generates 2 lines: pick 3 from A-01 (FEFO: earlier expiry first), pick 5 from B-03. • Pick confirmation for Bin A-01 qty=3: InventoryItem(A-01).qty_on_hand decremented by 3, ledger entry written • Short pick: picker finds only 2 in Bin A-01 (system said 3). picked_qty=2, pick_status=SHORT_PICKED. Remaining 1 unit backorder created. • Fulfillment from MIR auto-creates with source_type=MIR, inheriting variant_ids from MIR lines • Priority URGENT fulfillments appear at top of the pick queue • After all lines picked: fulfillment status transitions to PICKED
Acceptance	• FEFO-directed pick locations correct across multiple bins • Short-pick handling creates backorder without blocking the rest of the order • Ledger entries written per pick line in same transaction as stock decrement

WL-008**High****3d****Sec 8.3, 8.4****Wave picking, packing station workflow, and carton tracking**

Build PickWaves for batch picking multiple orders together, the pack station workflow recording carton contents with weight/dimensions, and carton status tracking.

Development tasks	Test cases (with test data)
-------------------	-----------------------------

• PickWaves entity: wave_id, wave_number (WAVE-YYYY-NNNNN), warehouse_id, status (OPEN/RELEASED/IN_PROGRESS/COMPLETED), fulfillment_ids (JSON array), created_at. In logistics_schema. • POST /api/pick-waves: create wave from selected fulfillment IDs. System consolidates identical variants across orders into wave pick lines (e.g. Order A needs 5 of X, Order B needs 3 of X = wave line for 8 of X). • POST /api/pick-waves/{id}/release: releases wave for picking. Individual pick lines assigned to pickers. • After wave picking, sort step: POST /api/pick-waves/{id}/sort assigns picked quantities back to individual fulfillments for packing. • Cartons entity (11 fields per FSD Sec 8.4) and CartonLines entity (5 fields). EF migration: AddPackingCartons • POST /api/fulfillments/{id}/pack: create carton, scan items into carton (each scan adds a CartonLine), record weight and dimensions. Multiple cartons per fulfillment supported. • POST /api/cartons/{id}/seal: finalize carton, print carton label with contents list and barcode. Status = SEALED. • React: Pack Station screen showing picked fulfillments ready for packing. Packer scans items into cartons, system validates scanned variant matches the fulfillment line.	• TEST DATA: Wave WAVE-001 groups FUL-001 (8 Dell i7) and FUL-002 (4 Dell i7 + 10 Cement). Wave pick list shows: Dell i7 x 12 (consolidated), Cement x 10. • After wave pick, sort correctly assigns: 8 Dell i7 back to FUL-001, 4 Dell i7 to FUL-002, 10 Cement to FUL-002 • Pack FUL-001: scan 8 Dell i7 into CTN-001. Weight=12.5kg. Seal. Carton label printed with barcode. • Scanning a variant not in the fulfillment returns error: 'This item is not in this order' • Multiple cartons: FUL-002 packed into CTN-002 (4 laptops) and CTN-003 (10 cement bags) due to weight limits • Wave with zero fulfillments returns 400 on creation
Acceptance	• Wave consolidation correctly merges identical variants across orders • Sort correctly de-consolidates back to individual orders • Carton tracking records exact contents per box

Epic 6 - Shipping, 3PL, Drop Ship (#12-14)

Multi-carrier shipping integration, 3PL inventory ownership, and drop-ship fulfillment.

WL-009**High****3d****Sec 9****Carrier framework, rate shopping, label generation, and tracking**

Build the carrier integration framework with adapter pattern, rate comparison API, shipping label generation, and tracking number capture/polling.

Development tasks	Test cases (with test data)
• Carriers entity (7 fields per FSD Sec 9.1) in logistics_schema. EF migration: AddCarriers • IShippingService interface: GetRates(origin, dest, weight, dims), GenerateLabel(carrierId, shipment), GetTracking(carrierId, trackingNo) • ICarrierAdapter interface implemented per carrier. Ship with adapters: TCSAdapter, LeopardsAdapter, FedExAdapter, DHLAdapter, ManualCarrierAdapter (for carriers without API - manual tracking entry). • GET /api/shipping/rates?fulfillmentId={id}: queries all active carriers, returns comparison table: carrier, estimated cost, estimated days • POST /api/shipping/labels: generate label for selected carrier. Returns PDF or ZPL. Stores tracking_number on the Carton entity. • GET /api/shipments/{fulfillmentId}/tracking: aggregates tracking status from carrier API for all cartons in the fulfillment • POST /api/fulfillments/{id}/ship: marks fulfillment as SHIPPED, sets shipped_date, writes TRANSFER_OUT ledger if applicable • React: Ship Station screen showing packed fulfillments. Rate comparison table. Ship button with carrier selection.	• TEST DATA: Carrier 'TCS', code='TCS', api_endpoint='https://api.tcs.com.pk', active. Carrier 'Manual', code='MANUAL', no API. • Rate query for 12.5kg shipment to Karachi returns rates from TCS and any other active carriers with cost and ETA • Label generation for TCS returns PDF with correct address, weight, and tracking barcode • Manual carrier: user enters tracking number manually, no API call made • Tracking query for TCS shipment returns status updates from TCS API • Fulfillment status transitions to SHIPPED after ship confirmation • Inactive carrier excluded from rate queries
Acceptance	• Adapter pattern allows adding new carriers without modifying core shipping logic • Rate comparison returns all active carriers for informed selection • Tracking aggregates correctly across multiple cartons

WL-010**Medium****2d****Sec 10, 11****3PL inventory ownership and drop-ship fulfillment**

Add owner-based inventory segregation for 3PL warehousing and implement drop-ship fulfillment that auto-generates supplier POs with customer addresses.

Development tasks	Test cases (with test data)
-------------------	-----------------------------

• InventoryOwners entity: owner_id, organization_id, owner_name, owner_type (SELF/CLIENT/CONSIGNMENT_VENDOR), contact_info, billing_terms. EF migration: AddInventoryOwners • Add owner_id (FK, default = org's SELF owner) to InventoryItems. Global query filter: optionally filter by owner_id when owner context is set. • All stock queries can filter by owner: GET /api/inventory?ownerId={id} • 3PL billing report: GET /api/reports/3pl-billing?ownerId={}.={} - storage days, transaction counts, charges • Drop ship: on FulfillmentOrder with fulfillment_type=DROP_SHIP, auto-generate a PO to the preferred supplier (from VariantSuppliers) with ship_to_address = fulfillment's ship_to_address. PO carries drop_ship_fulfillment_id. • Drop ship PO follows standard approval workflow. Supplier's tracking number captured on PO and propagated to fulfillment. • No InventoryItem movement for drop-shipped items - stock never enters the org's warehouse	• TEST DATA: Owner 'Client ABC' (type=CLIENT). 20 units of Cement owned by Client ABC stored in Main Warehouse. Org's own Cement = 80 units. • GET /api/inventory?ownerId=ClientABC returns only the 20 units. Default query (no owner filter) returns all 100. • 3PL billing for Client ABC: 20 units * 30 days = 600 storage-days at configured rate • Drop ship: FUL-003 type=DROP_SHIP for 5 Dell i7 to customer address. System auto-creates PO-2026-00099 to Lenovo Distributors with ship_to = customer address. • Drop ship: no InventoryItem qty_on_hand changes at any warehouse • Drop ship PO approval follows the standard PO workflow (Addendum 16)
Acceptance	• Owner-based segregation prevents cross-owner stock mixing • 3PL billing accurately calculates storage and transaction charges • Drop ship auto-PO correctly bypasses warehouse inventory

Epic 7 - Transfer Orders (#15 Enhancement)

Formal TransferOrder document with approval workflow and ship/receive lifecycle.

WL-011**High****3d****Sec 12****TransferOrders with approval workflow and ship/receive lifecycle**

Create TransferOrders and TransferOrderLines entities. Integrate with Addendum 16 workflow engine. Implement ship (source decrement) and receive (destination increment) with variance tracking.

Development tasks	Test cases (with test data)
• TransferOrders entity (12 fields per FSD Sec 12.1) and TransferOrderLines entity (7 fields per Sec 12.2) in warehouse_schema. EF migration: AddTransferOrders • Seed WorkflowDefinition: interface_code='TRANSFER_ORDER' with configurable approval chain • POST /api/transfer-orders: create DRAFT. POST /submit: enters workflow. On APPROVED: source warehouse can ship. • POST /api/transfer-orders/{id}/ship: validates status=APPROVED. For each line: decrement InventoryItem.qty_on_hand at source warehouse, write TRANSFER_OUT ledger entry. Set shipped_qty, shipped_date. Status = IN_TRANSIT. • POST /api/transfer-orders/{id}/receive: destination warehouse confirms receipt. For each line: increment InventoryItem.qty_on_hand at destination, write TRANSFER_IN ledger entry. Set received_qty, received_date. Status = RECEIVED. • Variance: if received_qty != shipped_qty for any line, flag for investigation. Variance recorded but does not block receipt. • MasterProductLedger entries: source_type=WAREHOUSE, source_name=source, destination_type=WAREHOUSE, destination_name=destination, both referencing the same transfer_number	• TEST DATA: TO-2026-00001 from Main Warehouse to Site Store: 20 units of Cement variant. Approved. Shipped 20. Received 18 (2 damaged in transit). • Ship: Main WH qty_on_hand decremented by 20. TRANSFER_OUT ledger entry written. • Receive: Site Store qty_on_hand incremented by 18. TRANSFER_IN ledger entry written. Variance = -2 flagged. • Transfer order for same warehouse (from=to) returns 400 • Shipping a non-APPROVED transfer returns 422 • Receiving a non-IN_TRANSIT transfer returns 422 • MasterProductLedger shows 2 entries for the same transfer: OUT from Main WH, IN to Site Store
Acceptance	• Full lifecycle: DRAFT > SUBMITTED > APPROVED > SHIPPED > IN_TRANSIT > RECEIVED • Variance between shipped and received tracked without blocking receipt • Ledger entries correctly pair source and destination for audit

Epic 8 - Mobile Warehouse App (#16, #17)	React Native / PWA mobile app for warehouse floor operations and field inventory with offline capability.
---	---

WL-012	High	4d	Sec 13	Mobile app: GRN receiving, put-away, picking, and stock lookup with barcode s
---------------	-------------	-----------	---------------	--

Build the core mobile warehouse app (React Native or PWA) with barcode camera scanning for the 4 most critical warehouse operations: GRN receiving, bin put-away, order picking, and instant stock lookup.

Development tasks	Test cases (with test data)
• React Native project setup (or PWA with service worker). Same JWT auth as web app. Biometric re-auth for shared devices. • Barcode scanning: integrate device camera via react-native-camera or Web Barcode Detection API. Decode Code128, EAN-13, QR. • GRN Receiving screen: scan PO barcode to load PO lines. Scan each item barcode to confirm received variant. Enter quantities. Submit to existing GRN API. • Put-away screen: after GRN, scan item barcode, then scan destination bin barcode. Calls PUT /api/inventory/{variantId}/bin-assignment to record the bin location. • Pick screen: load assigned pick list from GET /api/fulfillments/{id}/pick-lines. Scan bin barcode to confirm location, scan item barcode to confirm variant, enter qty. Submit via POST /api/fulfillments/{id}/pick. • Stock lookup: scan any barcode, calls GET /api/variants/lookup?barcode={code}, displays real-time stock levels across all warehouses. • Push notifications: Firebase Cloud Messaging for new pick assignments, wave releases, transfer arrivals. • Bluetooth printer pairing: print labels from the mobile app via Bluetooth-connected portable Zebra/Brother printers.	• Scan barcode '8901234567892' on stock lookup: displays 'Dell Latitude i7/16/512, Main WH: 5 units, Site Store: 0 units' • GRN receiving: scan PO barcode loads correct PO lines. Scan item barcode '8901234567890' correctly matches Dell i5 variant on PO line 1. • Scan item barcode NOT on the PO shows warning: 'This item is not on this PO' • Put-away: scan item then scan bin 'BIN-A-01' records the bin assignment • Pick: scan bin 'BIN-A-01' confirms correct location. Scan item barcode confirms correct variant. Short-pick entry allowed. • Push notification received when a new pick list is assigned to the mobile user • Bluetooth print: successfully prints a product label on a paired Zebra ZQ520
Acceptance	• All 4 core operations work end-to-end with barcode scanning • Push notifications delivered reliably • Bluetooth printing functional with standard portable printers

WL-013	Medium	3d	Sec 13.1 (#17)	Mobile app: cycle counting, transfer receiving, and offline field inventory mode
---------------	---------------	-----------	-----------------------	---

Add cycle counting and transfer receiving to the mobile app. Implement offline-capable field inventory mode for truck/offsite stock management with sync-on-reconnect.

Development tasks	Test cases (with test data)
-------------------	-----------------------------

• Cycle count screen: load assigned count sheet. Scan bin barcode, scan items, enter counted quantities. Submit to POST /api/cycle-counts/{id}/lines/{lineId}/count. • Transfer receiving screen: scan transfer order barcode, scan received items, confirm quantities. Submit to POST /api/transfer-orders/{id}/receive. • Offline mode: when device detects no network, switch to offline mode. All scans and entries saved to local SQLite (React Native) or IndexedDB (PWA). • Sync queue: pending transactions queued locally with timestamps. On reconnect, POST /api/mobile/sync sends queued transactions in order. Server-wins conflict resolution for stock quantities. • Offline stock cache: last-known stock levels cached locally for stock lookup in offline mode. Displayed with 'Offline - data as of {timestamp}' warning. • Field inventory: dedicated 'Truck Stock' mode where the user registers items loaded on a truck, tracks consumption in the field, and syncs the remaining stock back to the server.	• Cycle count in offline mode: 5 lines counted offline. On reconnect, all 5 submitted successfully to server. • Conflict: user counts 10 offline, but another user adjusted stock to 8 online during the offline period. Server-wins: stock = 8, offline count recorded as variance for review. • Transfer receiving offline: scan 18 of 20 items. On reconnect, transfer received with received_qty=18. • Field inventory: load 50 Cement bags on truck. Consume 30 in field. Sync back: 20 remaining reflected in warehouse stock. • Offline mode indicator clearly shown on all screens when device is offline • Sync queue processes transactions in chronological order, not in parallel
Acceptance	• Offline mode genuinely works without network for all supported operations • Sync-on-reconnect handles conflicts without data loss • Field inventory mode accurately tracks truck/offsite consumption

3. Sequencing Notes

Epics are largely independent modules that can be parallelized. Key dependencies noted below.

Dependency	Reason
WL-001 before WL-012	Mobile app barcode scanning uses the label/generation service.
WL-003 before WL-013	Mobile cycle counting calls the cycle count API from WL-003.
WL-005 before WL-006	Forecast reorder recommendations reference VariantSuppliers data.
WL-007 before WL-008	Wave picking and packing build on the fulfillment/pick entities.
WL-007 before WL-009	Shipping operates on packed fulfillments from the pick/pack flow.
WL-011 before WL-013	Mobile transfer receiving calls the TransferOrder API.
Epics 1-4 can fully parallel	Barcode, expiry, counting, assets, multi-vendor have no cross-dependencies.
Epics 5-6 (Pick/Pack/Ship) are the critical path	Fastest new capability; start early if warehouse fulfillment is the priority.

- End of Document -